Trabalho Prático de Planejamento Automático: Dial-a-Ride Problem

Felipe Escobar Koizimi

Inteligência Artificial – Prof. Jomi F. Hübner

A prefeitura de Joinville tem um serviço de transporte porta a porta para pessoas com deficiência e mobilidade reduzida. A pessoa faz o pedido na véspera, dizendo de onde sai, para onde vai e a que horas. No início do dia, com todos os pedidos já conhecidos, é preciso decidir qual veículo atende cada pedido, em que ordem e em que horário.

O problema foi escrito em PDDL em quatro versões, da mais simples à mais completa, e resolvido com solvers de prateleira. Foram geradas oito instâncias, de 1 a 10 pedidos. O Metric-FF resolveu todas elas nas três versões com custo, 24 execuções, sempre em segundos.

Seção	Conteúdo
1	A solução em PDDL
2	As extensões realizadas
3	A qualidade das soluções dos solvers
4	Avaliação do planejamento automático para o problema
5	Conclusões

O código, as instâncias e os dados estão em https://github.com/escobar-felipe/tp-darp-pddl.

---

1. A solução em PDDL

1.1 O estado

Há três tipos de objeto: veículo, passageiro e lugar. O estado é formado por cinco predicados:

```
(vehicle-at   ?v - vehicle   ?l - place)   onde esta cada veiculo
(passenger-at ?p - passenger ?l - place)   onde esta cada passageiro na rua
(onboard      ?p - passenger ?v - vehicle) quem esta dentro de qual veiculo
(served       ?p - passenger)              quem ja foi entregue
(destination  ?p - passenger ?l - place)   para onde o passageiro quer ir
```

Os quatro primeiros mudam ao longo do plano. O `destination` é o pedido em si: entra no estado inicial e nunca muda.

Separar o pedido da posição do passageiro tem um motivo. O objetivo do plano é `(served ana)`, e não `(passenger-at ana centro)`. Queremos que a pessoa tenha sido transportada, e não só que ela esteja no centro, onde ela poderia já estar desde o começo.

No estado inicial ficam os veículos na garagem, a posição e o destino de cada passageiro, e os tempos entre os lugares. O objetivo é `(served ?p)` para todos os pedidos: ou se atende todo mundo, ou o problema não tem solução. É o que o enunciado permite assumir nesta etapa.

1.2 As ações

Três ações dão conta do caso básico:

```
drive   (?v ?from ?to)   move o veiculo, se ele estiver em ?from
pickup  (?p ?v ?l)       embarca, se passageiro e veiculo estiverem em ?l
dropoff (?p ?v ?l)       desembarca, se ?l for o destino do pedido
```

A pré-condição do `dropoff` é o que garante o atendimento correto:

```
:precondition (and (onboard ?p ?v)
                   (vehicle-at ?v ?l)
                   (destination ?p ?l))
```

Sem a última linha, o planejador poderia largar o passageiro em qualquer lugar e marcar o pedido como atendido, porque o `served` sairia do mesmo jeito. Com ela, o pedido só é dado como atendido no lugar que a pessoa pediu. Isso se verifica lendo a regra, sem precisar rodar nada.

O `pickup` não exige que o veículo esteja vazio. Então um veículo pode levar várias pessoas ao mesmo tempo, que é o ponto do DARP: as corridas são compartilhadas, não uma de cada vez.

1.3 Custo e critério de otimização

Cada deslocamento tem uma duração. O tempo entre dois lugares é um número dado no estado inicial, e o `drive` vai somando:

```
(:functions (travel-time ?from ?to - place) (total-cost) - number)

(:action drive
  :effect (and ... (increase (total-cost) (travel-time ?from ?to))))

(:metric minimize (total-cost))
```

O critério de otimização é o tempo total que a frota passa rodando. É o custo de quem opera o serviço: combustível, hora de motorista, veículo ocupado. E é a única medida que existe nas quatro versões, o que permite comparar uma com a outra.

Embarcar e desembarcar não custam nada. Em qualquer plano válido, cada passageiro embarca uma vez e desembarca uma vez, então um tempo fixo de parada seria o mesmo número somado a todos os planos e não mudaria qual rota é a melhor. Isso mudaria se as paradas demorassem tempos diferentes entre si, e nesse caso o tempo de parada precisaria entrar na conta.

Esse critério não mede o tempo que o passageiro passa dentro do veículo. Economizar para o operador significa juntar corridas, e juntar corridas faz o passageiro dar voltas antes de chegar. A literatura de DARP trata isso limitando o quanto a viagem de cada um pode ser desviada. Aqui ficou só o lado do operador, que é uma escolha e uma limitação do modelo.

---

2. As extensões realizadas

As duas extensões couberam no mesmo modelo, cada uma com poucas linhas a mais, sem mexer na estrutura do que já estava escrito.

2.1 Vários veículos, com capacidades diferentes

Entram duas funções, e a capacidade passa a limitar o embarque:

```
(:functions (capacity ?v - vehicle) (seats-used ?v - vehicle) ...)

(:action pickup
  :precondition (and ... (< (seats-used ?v) (capacity ?v)))
  :effect       (and ... (increase (seats-used ?v) 1)))

(:action dropoff
  :effect       (and ... (decrease (seats-used ?v) 1)))
```

Com isso o planejador decide também **qual** veículo pega cada pedido, que é a parte de atribuição do DARP. No exemplo da pasta `v2-veiculos`, uma van de três lugares e um carro de um lugar atendem quatro pedidos.

2.2 Hora de saída e espera

Cada veículo ganha um relógio, que anda junto com os deslocamentos, e cada pedido ganha a hora a partir da qual o passageiro pode embarcar:

```
(:functions (time ?v - vehicle) (ready-at ?p - passenger) ...)

(:action drive   :effect (and ... (increase (time ?v) (travel-time ?from ?to))))
(:action pickup  :precondition (and ... (>= (time ?v) (ready-at ?p))))
```

Se o veículo chega antes da hora, ele espera, e para isso entra a quarta ação:

```
(:action wait
  :parameters   (?v - vehicle ?p - passenger ?l - place)
  :precondition (and (vehicle-at ?v ?l) (passenger-at ?p ?l)
                     (< (time ?v) (ready-at ?p)))
  :effect       (assign (time ?v) (ready-at ?p))
)
```

A espera pula direto para a hora do pedido, em vez de andar de minuto em minuto. Esperar 20 minutos ou esperar 1 minuto vinte vezes dá no mesmo para o problema, e o salto evita encher o plano de passos de espera. Com ele, há no máximo uma espera antes de cada embarque.

Uma limitação para registrar: cada veículo tem o seu relógio, não existe relógio geral. Em PDDL clássico as ações acontecem uma depois da outra, então o `(time ?v)` conta o tempo daquele veículo desde que saiu da garagem. Isso está correto para cada rota separada, mas não permite escrever regras que liguem dois veículos no mesmo instante. Para isso seria preciso PDDL temporal, com `:durative-actions`.

---

3. A qualidade das soluções dos solvers

3.1 Como foi testado

O `scripts/gerar.py` sorteia oito instâncias, de 1 a 10 pedidos, e escreve cada uma nos três formatos (v1, v2 e v3), com os mesmos lugares, tempos e pedidos. Assim o custo é comparável tanto entre solvers quanto entre as modelagens.

Instância	Pedidos	Veículos	Lugares
p1	1	1	3
p2	2	1	4
p3	3	1	5
p4	4	2	6
p5	5	2	7
p6	6	2	8
p7	8	3	10
p8	10	3	12

Os lugares ganham coordenadas num mapa de 20 por 20 e o tempo entre dois lugares é a distância entre eles, arredondada, o que garante que ir direto nunca seja pior do que passar por um terceiro lugar.

Foram testados quatro solvers, todos pelas imagens Docker usadas nos exemplos da disciplina:

Nome no script	Solver	Tipo de busca
`ff`	Metric-FF	para no primeiro plano bom, aceita números
`lama`	Fast Downward, `lama-first`	para no primeiro plano bom, só custos
`opt`	Fast Downward, `seq-opt-lmcut`	devolve o plano de menor custo, só custos
`optic`	OPTIC	para no primeiro plano bom, números e tempo

O `opt` entrou para servir de referência: sem ele, não haveria como saber se um plano de custo 45 é bom ou ruim.

3.2 Os planos foram encontrados

O Metric-FF resolveu as oito instâncias nas três versões com custo, 24 execuções, todas em segundos. Todos os planos entregam os passageiros no destino certo, respeitam a capacidade dos veículos e respeitam a hora de saída dos pedidos. Essas três condições estão nas pré-condições das ações, e um plano só é devolvido se elas valerem em cada passo.

Para o tamanho de instância que o trabalho testou, a modelagem resolve o problema de ponta a ponta: descreve o pedido, escolhe o veículo, monta a rota e respeita o horário.

3.3 O custo dos planos

A tabela abaixo é da v1, a única versão em que existe uma referência do melhor plano. Os custos saem de `results/avaliacao.csv`.

Instância	Pedidos	Metric-FF	`lama-first`	`seq-opt-lmcut` (melhor)
p1	1	15	15	15
p2	2	17	16	16
p3	3	48	61	37
p4	4	53	55	49
p5	5	69	112	49
p6	6	55	136	55
p7	8	111	122	56
p8	10	108	119	não terminou em 20s

O Metric-FF chegou ao melhor plano em p1 e p6, e ficou a 6% e 8% dele em p2 e p4. Nas três instâncias restantes a diferença foi maior, com destaque para p7, onde o plano custa 111 contra 56 do melhor. O `lama-first` acompanhou nas duas primeiras e se afastou mais nas demais.

A leitura é que os solvers que param no primeiro plano bom entregam rota utilizável em todas as instâncias, mas sem garantia de ser a mais barata, e a diferença varia de instância para instância. Quando o custo importa, o `seq-opt-lmcut` resolve e prova o melhor plano até 8 pedidos. Na instância de 10 ele passou do limite de 20 segundos usado na campanha, e um limite maior provavelmente o levaria adiante.

Uma observação sobre os tempos registrados no CSV: cada execução sobe um container Docker emulado, o que custa de 1 a 6 segundos antes de o solver começar. Eles servem para comparar uma instância com outra, não como tempo puro de busca.

3.4 O que cada solver aceita

Versão	O que usa	Metric-FF	Fast Downward	OPTIC
v0, v1	custo fixo, dado no início	sim	sim	sim
v2	número que muda no plano (`seats-used`)	sim	não	sim
v3	`assign` na ação `wait`	sim	não	não

O Fast Downward aceita custo de ação, que é um número lido no começo e nunca mais alterado. Como a capacidade precisa de um número que sobe e desce durante o plano, a v2 e a v3 ficam fora dele. O OPTIC aceita números, mas o parser dele não aceitou o `assign` da ação `wait`.

O efeito prático é que o Metric-FF foi o solver que acompanhou o modelo até a última versão, e é ele quem responde nas mais completas. Se for preciso ter de volta a referência de melhor plano nessas versões, a capacidade pode ser escrita sem números, com uma cadeia de predicados do tipo `(capacity-predecessor ?n1 ?n2)`, como fazem os domínios clássicos de logística. O modelo fica menos legível e o Fast Downward volta a funcionar.

---

4. Avaliação do planejamento automático para o problema

A técnica se saiu bem no que este trabalho precisou dela. O modelo inteiro tem quatro ações e cinco predicados, e cobre pedido, rota, escolha de veículo, capacidade e horário. Escrito como programa, isso seria bem mais código, e boa parte dele seria a busca, que aqui vem pronta.

Mudar uma regra do serviço é mudar uma linha. A capacidade entrou como uma pré-condição e um efeito; o horário, como outra pré-condição e uma ação nova. As três versões com custo compartilham as mesmas ações, e nenhuma extensão obrigou a repensar o que já estava escrito. Para explorar variações de regra, que é o que se faz no início de um projeto desses, isso vale muito.

A correção também sai barata. A pré-condição do `dropoff` mostra, na própria regra, que ninguém é largado no lugar errado, e a do `pickup` mostra que a capacidade e o horário são respeitados. Em um programa comum, cada uma dessas garantias viraria um teste.

O limite aparece na qualidade do custo. O DARP é um problema de rota, e as heurísticas dos planejadores genéricos estimam o quanto falta contando fatos que ainda não foram alcançados. Em um problema de rota, o que custa é a ordem em que se visita os lugares, e todas as ordens alcançam os mesmos fatos no fim, então a heurística distingue pouco uma rota boa de uma ruim. Daí os solvers satisfatórios entregarem rota válida rapidamente mas variarem no custo, e o solver ótimo precisar de muito mais trabalho nas instâncias maiores.

O encaminhamento natural é usar o modelo PDDL como a descrição executável das regras do serviço, que é onde ele é forte, e tratar o custo como um segundo passo: com o solver ótimo nas instâncias em que ele fecha, ou conferindo contra o modelo a saída de um resolvedor de rotas especializado.

---

5. Conclusões

O DARP foi escrito em PDDL em quatro versões, de um veículo com um passageiro até uma frota com capacidades e horários de saída. O critério de otimização adotado foi o tempo total de deslocamento da frota. Foram geradas oito instâncias, de 1 a 10 pedidos, e testados quatro solvers sobre elas.

O que os dados sustentam:

1. A modelagem resolve o problema no intervalo testado. O Metric-FF encontrou plano em todas as 24 execuções das três versões com custo, em segundos, e todos os planos respeitam destino, capacidade e horário.

2. As extensões couberam no mesmo modelo. A capacidade e o horário entraram como pré-condições e efeitos, sem mexer na estrutura das ações.

3. Os solvers que param no primeiro plano bom nem sempre acham o mais barato. O Metric-FF chegou ao melhor plano em duas das sete instâncias com referência e ficou perto em outras duas. Quando o custo importa, o `seq-opt-lmcut` prova o melhor plano até 8 pedidos.

4. Cada extensão restringiu o conjunto de solvers aplicáveis, e o Metric-FF foi o que acompanhou o modelo até a última versão.